Run the test. At this point, one of two things will happen: the test will error because there's a missing dependency, or the test will fail because the code runs but spits out a different result. (Well, three things, if you include the remote possibility that the sales tax really is –300,000 dollars, in which case you might have bigger problems.)

Most of the time the test will raise an error because there is some object dependency you didn't know about, some value is not as expected, or you have otherwise disturbed the delicate balance of your legacy app's functionality. You'll need to figure out how to smooth things over. Often you'll have to create more objects. In this example, you might need to explicitly create product objects. The object chain can get unwieldy, which is okay at this point: the goal is to understand what's happening. If the code itself is unwieldy, let the test stand as a monument to things that need to be changed.

Eventually you'll run out of errors; the application spits out the sales tax, and the test has a normal validation failure—since, again, the answer probably isn't –300,000. At this point you insert the actual value into the test and declare victory. You now have some test coverage and a greater understanding of how the application fits together. It's time to move on to the next test, most likely trying the existing method in some other test case, such as one designed to trigger a bad response.

You don't care if the value for the sales tax is correct. Well, you care in the sense that calculating sales tax is important from a business perspective; however, from a test perspective you must assume that the code is correct so that you have a stable base when you start making changes because of all the bugs.

Prescription 54	When writing initial unit tests for legacy code, use the test to explore the code's behavior. Try to write a passing test without changing code.

Pry can be your friend and ally during this exploration process. The console is a great way to try some of these object interactions quickly. Once you figure things out in the console, transfer the commands to the test so they can be run repeatedly.

What Tool Should I Use? (Legacy Edition)

Taking over a legacy codebase has the side effect of clarifying tool decisions you might otherwise agonize over—if the previous coder used a tool and there's anything salvageable, use that tool. You don't want to be in the position of